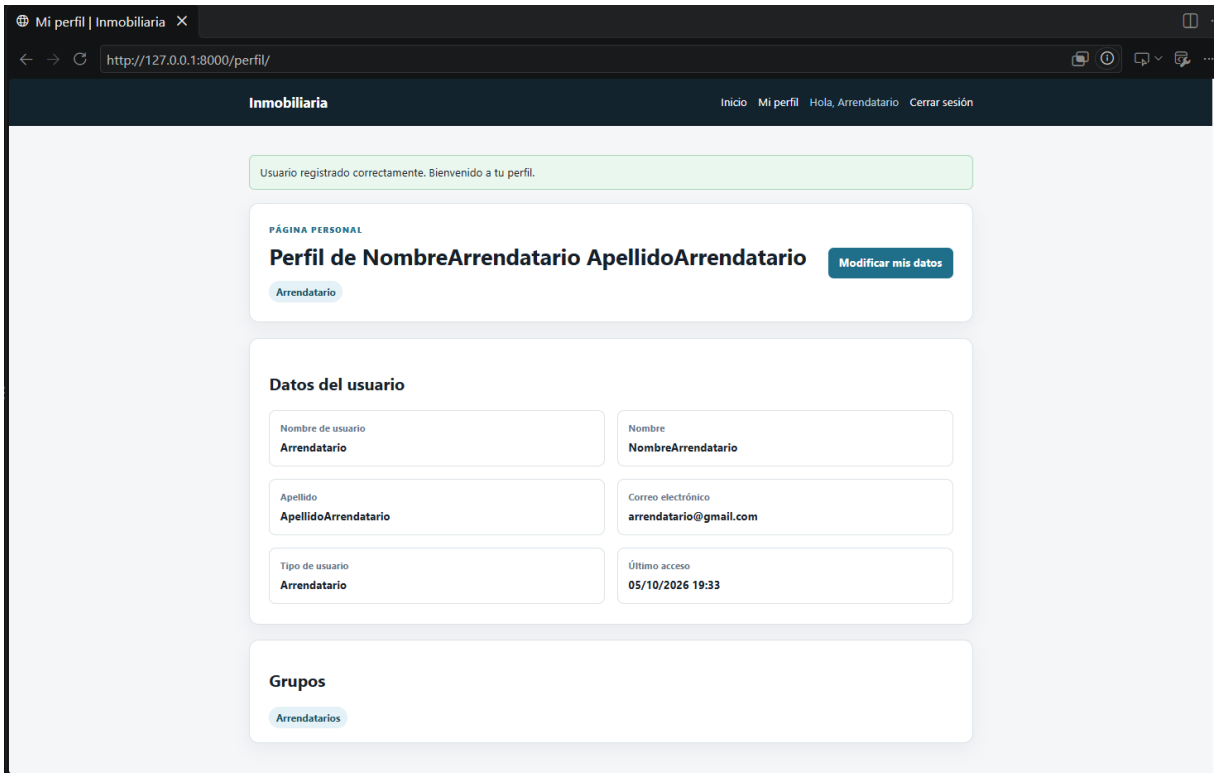
Informe Final del Proyecto

Aplicación web de arriendo de inmuebles con Django

Hito 5 - Informe de presentación del proyecto

Proyecto	Sitio web para publicar y revisar inmuebles disponibles para arriendo.
Stack	Python, Django, PostgreSQL, Django Auth, ORM y Admin.
Usuarios	Arrendatarios, Arrendadores y Administrador.
Producto final	Aplicación funcional con autenticación, perfiles, carga de datos, reportes y CRUD web.



Vista de perfil de usuario autenticado en el proyecto.

Índice de contenidos

1. Resumen ejecutivo
2. Objetivo y alcance del proyecto
3. Arquitectura general y patrón MVC/MVT en Django
4. Evolución por hitos
5. Características implementadas
6. Demostración funcional
7. Filtrado y reportes por comuna y región
8. Problemas encontrados y soluciones aplicadas
9. Conclusiones
10. Anexo de evidencias

1. Resumen ejecutivo

El proyecto desarrollado corresponde a una aplicación web de arriendo de inmuebles construida con Django. La solución permite administrar información geográfica, usuarios, grupos, permisos e inmuebles, además de habilitar flujos diferenciados para arrendatarios y arrendadores. El resultado final integra base de datos SQL, autenticación, perfiles de usuario, reportes y operaciones CRUD sobre el modelo Inmueble.

- Los arrendadores pueden publicar, editar y eliminar inmuebles propios.
- Los arrendatarios pueden revisar la oferta disponible.
- El administrador puede gestionar regiones, comunas, tipos de inmueble, usuarios y permisos desde el Admin de Django.
- La base de datos fue poblada con fixtures y consultada mediante ORM y SQL según los requerimientos de los hitos.

2. Objetivo y alcance del proyecto

El objetivo general fue construir progresivamente un sitio web que resolviera el caso de una empresa dedicada al arriendo de inmuebles. A través de los distintos hitos se avanzó desde la conexión inicial con PostgreSQL hasta una aplicación con usuarios, roles, perfiles y CRUD web.

- Diseñar modelos relacionales para regiones, comunas, tipos de inmueble e inmuebles.
- Configurar PostgreSQL como base de datos del proyecto.
- Implementar autenticación, login, logout y registro de usuarios.
- Crear perfiles diferenciados para arrendatarios y arrendadores.
- Publicar inmuebles desde formularios web usando el ORM de Django.
- Generar reportes por comuna y región a partir de datos almacenados.

3. Arquitectura general y patrón MVC/MVT en Django

Django trabaja con una adaptación del patrón MVC conocida habitualmente como MVT: Modelo, Vista y Template. En este proyecto, los modelos representan la capa de datos; las vistas reciben la petición, aplican reglas de negocio y consultan el ORM; los templates construyen la respuesta HTML para el usuario.

Modelo	Vista / Controlador	Template
Region, Comuna, TipoInmueble, Inmueble	registro, perfil, crear_inmueble, editar_inmueble, oferta_inmuebles	login.html, registro.html, perfil.html, inmueble_form.html, oferta_inmuebles.html
Persistencia en PostgreSQL	Consulta y validación con ORM	HTML para interacción del usuario

Esta separación permitió mantener el proyecto ordenado: los cambios de datos se concentran en modelos y formularios, la lógica de acceso queda en las vistas, y la presentación queda en templates reutilizables.

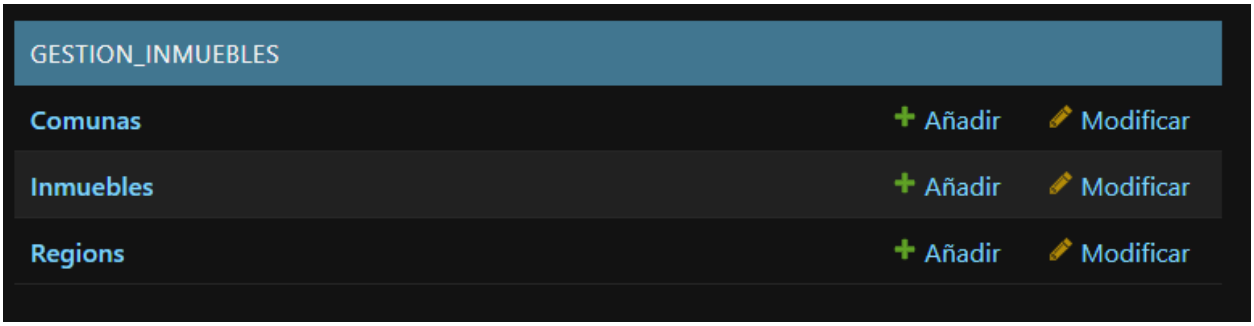
4. Evolución por hitos

Hito	Resultado principal	Evidencia técnica
Hito 1	Conexión Django + PostgreSQL y primeros modelos.	Migraciones, modelos y CRUD desde shell/ORM.
Hito 2	Admin de Django personalizado.	Modelos registrados y listados con búsqueda/filtros.
Hito 2B	Autenticación con Django Auth.	Registro, login, logout, grupos y permisos.

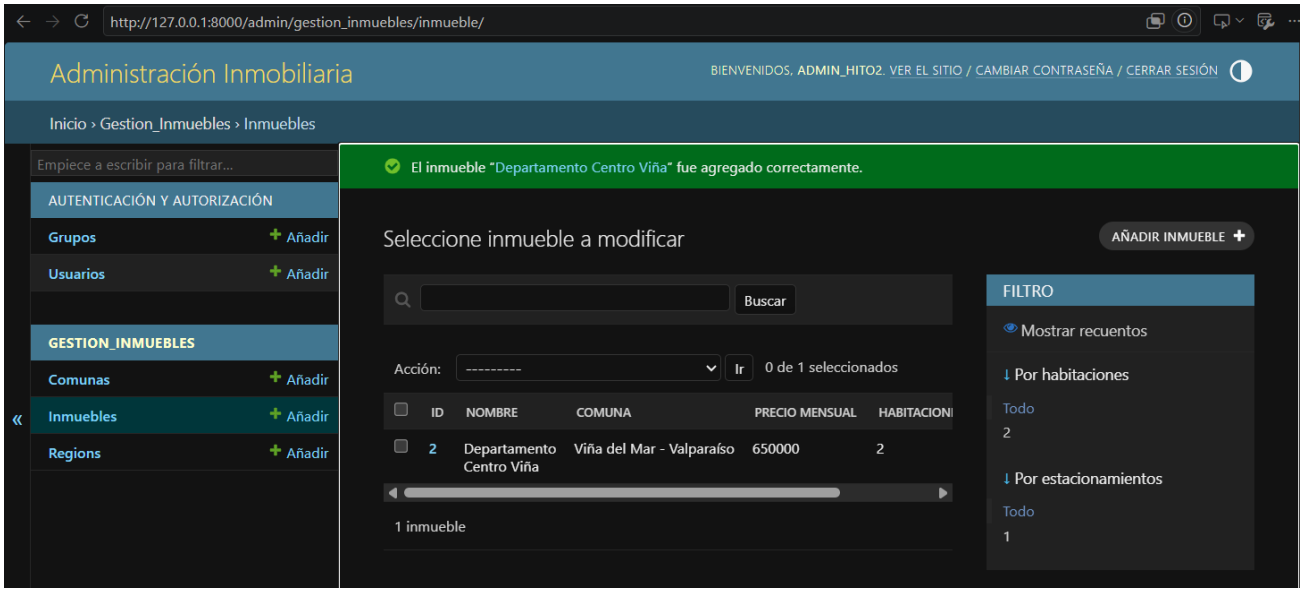
Hito 3	Carga de datos y reportes.	Fixtures con loaddata y scripts SQL a archivos .txt.
Hito 4	Perfiles de usuario.	Perfil de arrendatario/arrendador y edición de datos personales.
Hito 4B	CRUD web con ORM.	Crear, actualizar, eliminar y listar inmuebles desde la web.

5. Panel de administración

El panel de administración se configuró para que el equipo administrador pudiera gestionar modelos clave del negocio. Se registraron regiones, comunas, inmuebles y usuarios, con personalizaciones de búsqueda, filtros y columnas de listado.



Panel principal del Admin con modelos de gestión inmobiliaria.



Listado de inmuebles en Admin con búsqueda y filtros laterales.

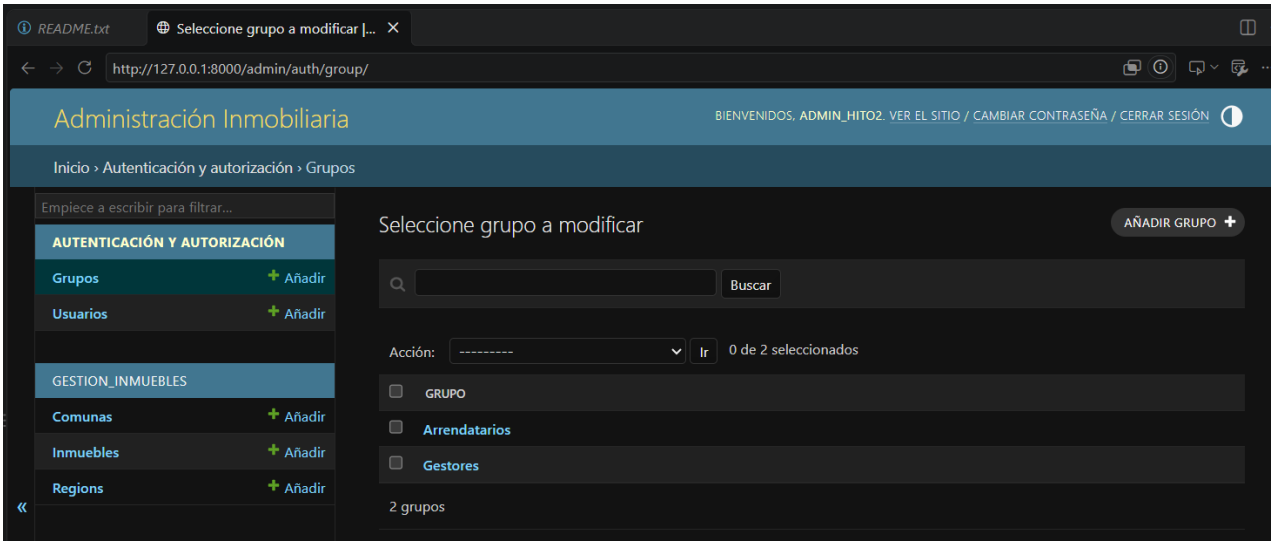
6. Autenticación, permisos y grupos

La autenticación se implementó con las herramientas incluidas en Django. Se habilitó registro de usuarios, inicio y cierre de sesión, y se configuraron grupos para diferenciar permisos entre tipos de usuario.

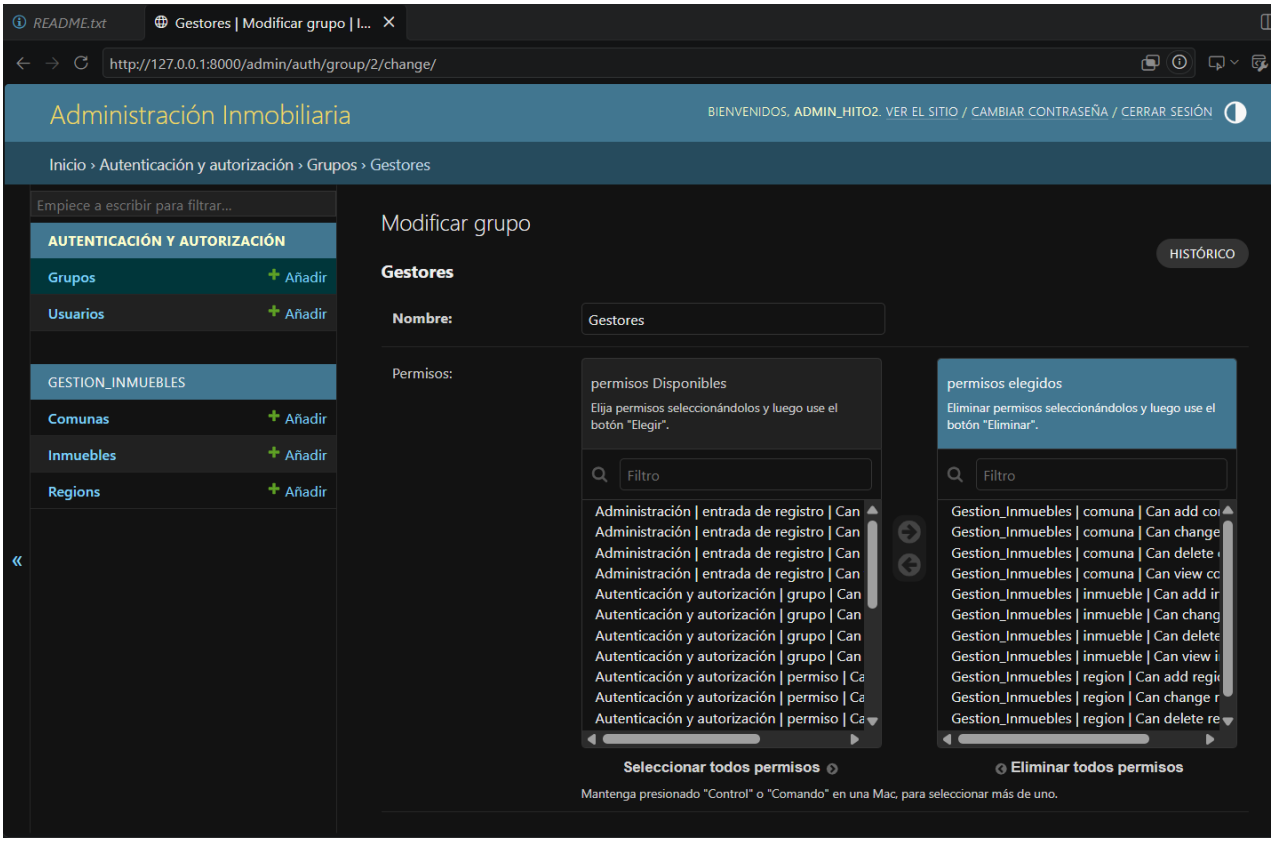
- Arrendatarios: usuarios que revisan la oferta disponible.
- Arrendadores: usuarios que pueden administrar sus propios inmuebles.
- Administrador: usuario con acceso al panel de administración.



Formulario de registro de usuarios.



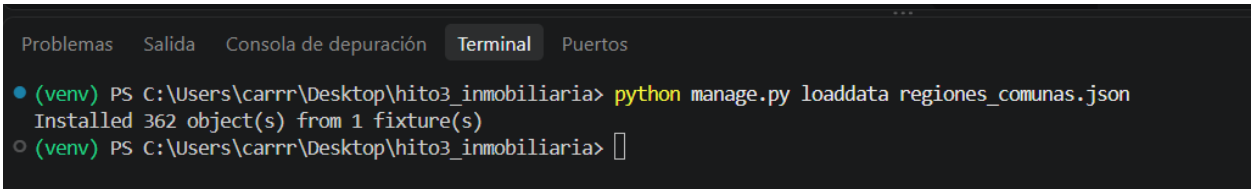
Grupos de usuarios creados para separar responsabilidades.



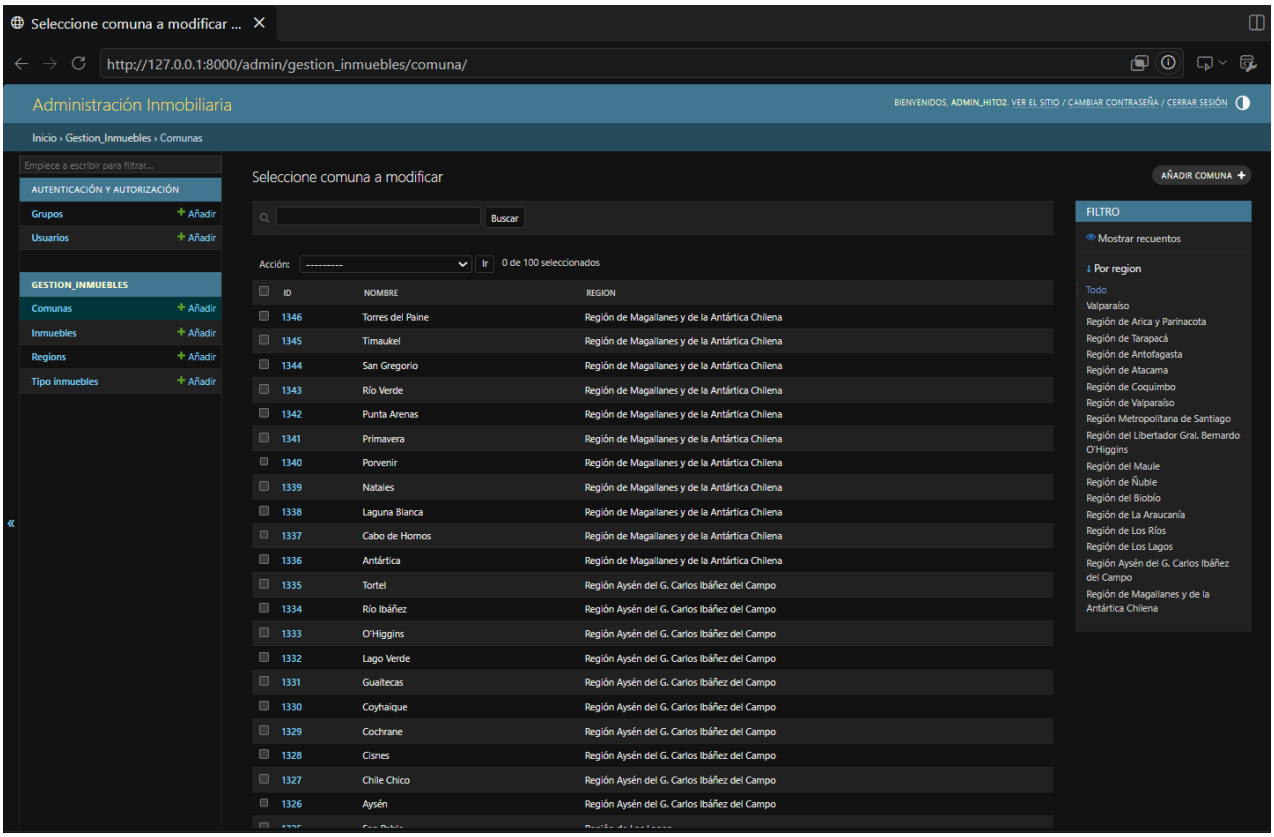
Permisos asignados al grupo Gestores en el Admin.

7. Carga de datos y recuperación de información

En el Hito 3 se poblaron datos iniciales mediante fixtures ejecutadas con loaddata. Esto permitió disponer de regiones, comunas, tipos de inmueble, usuarios e inmuebles de prueba para reportes y navegación.



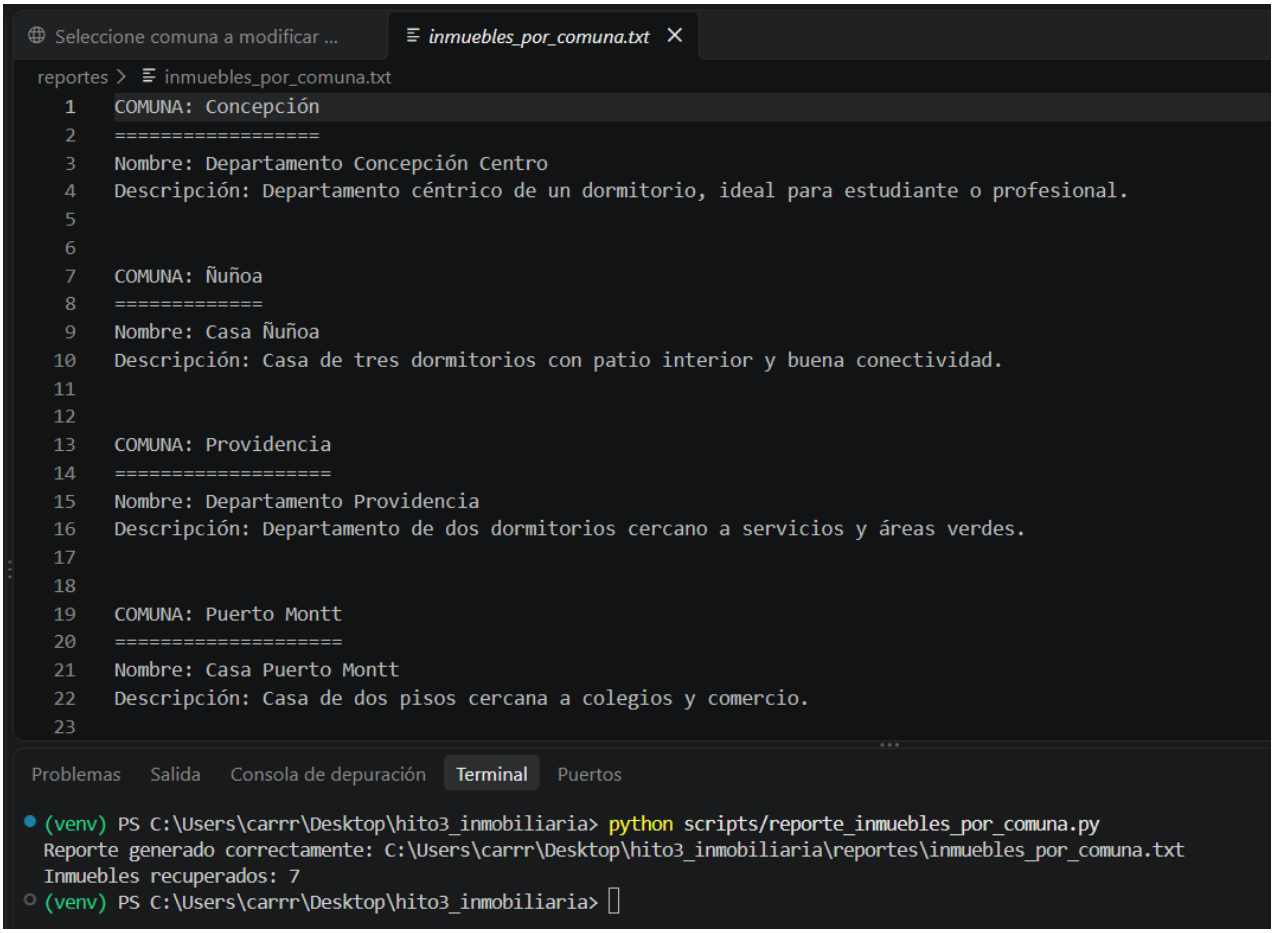
Ejecución de fixture para regiones y comunas.



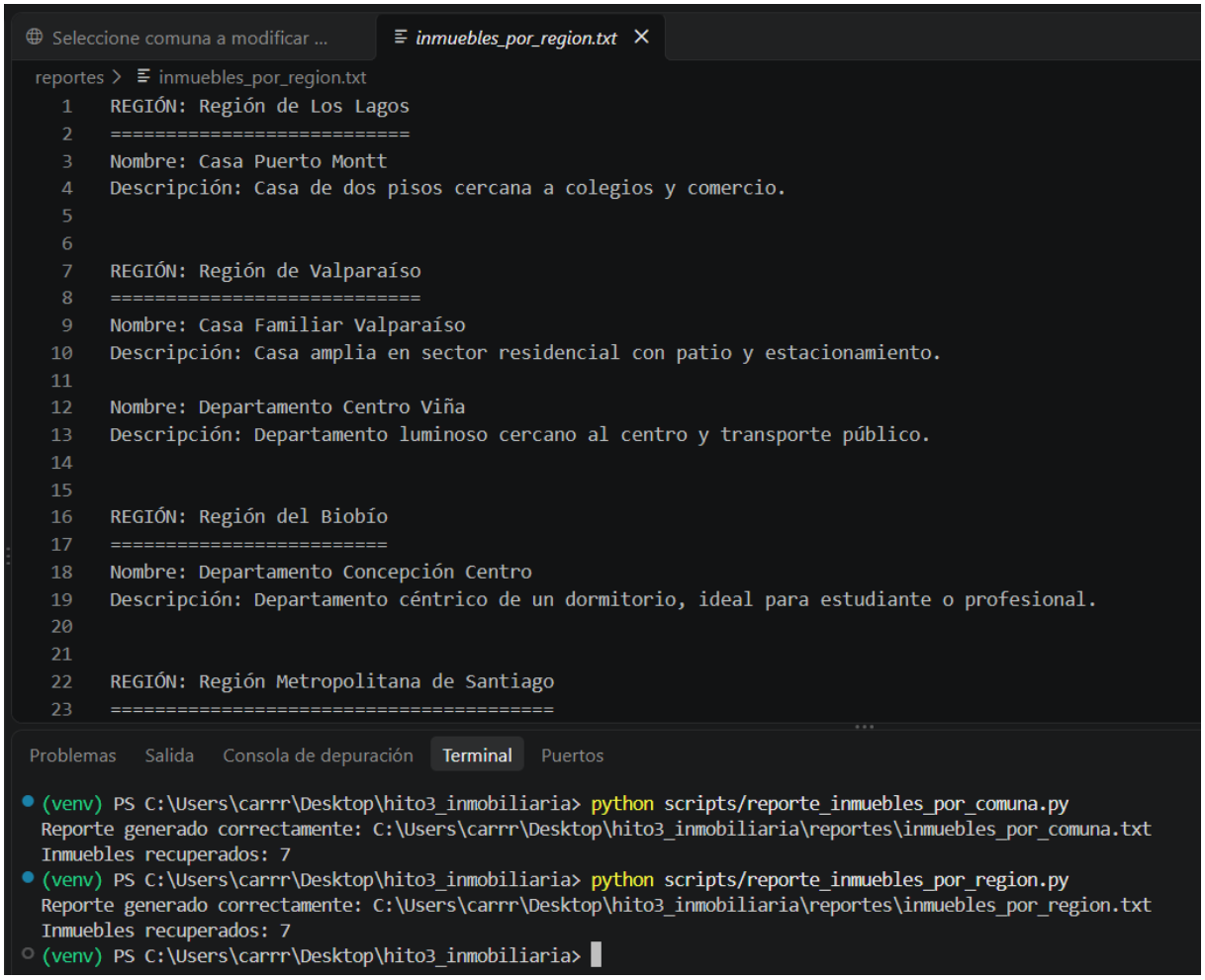
Verificación de comunas cargadas en el sistema.

8. Reportes por comuna y región

Además del uso del ORM, se crearon scripts Python que inicializan Django, ejecutan consultas SQL y guardan resultados en archivos de texto. Estos reportes permiten revisar inmuebles separados por comuna y por región.



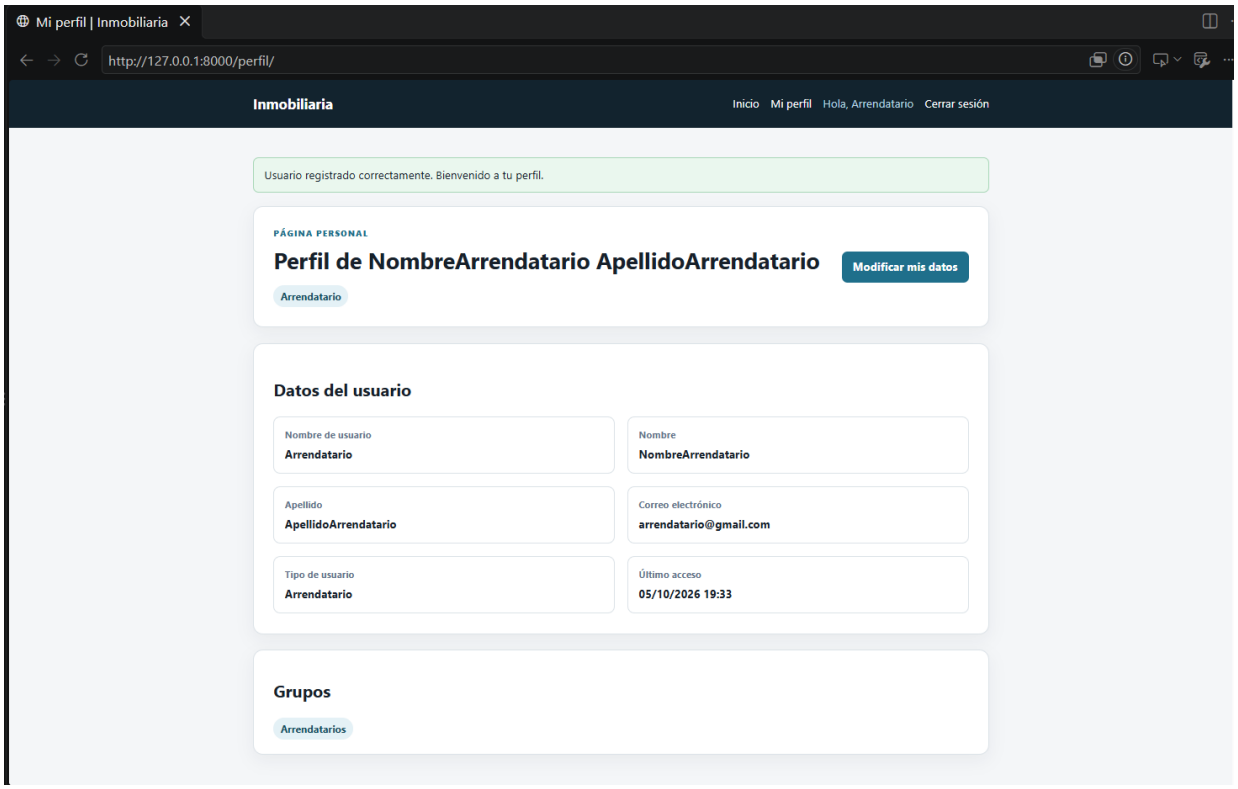
Reporte de inmuebles separado por comunas.



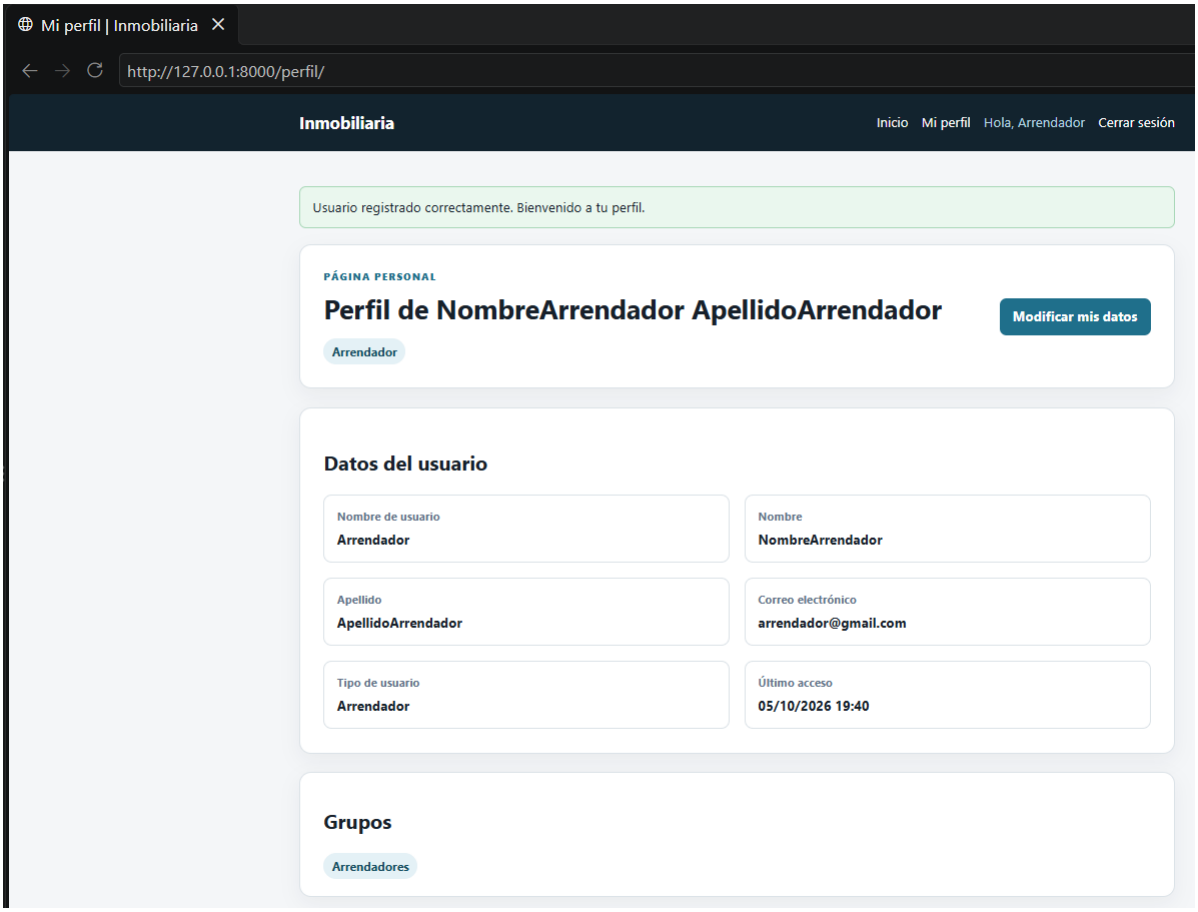
Reporte de inmuebles separado por regiones.

9. Perfiles de Arrendatarios y Arrendadores

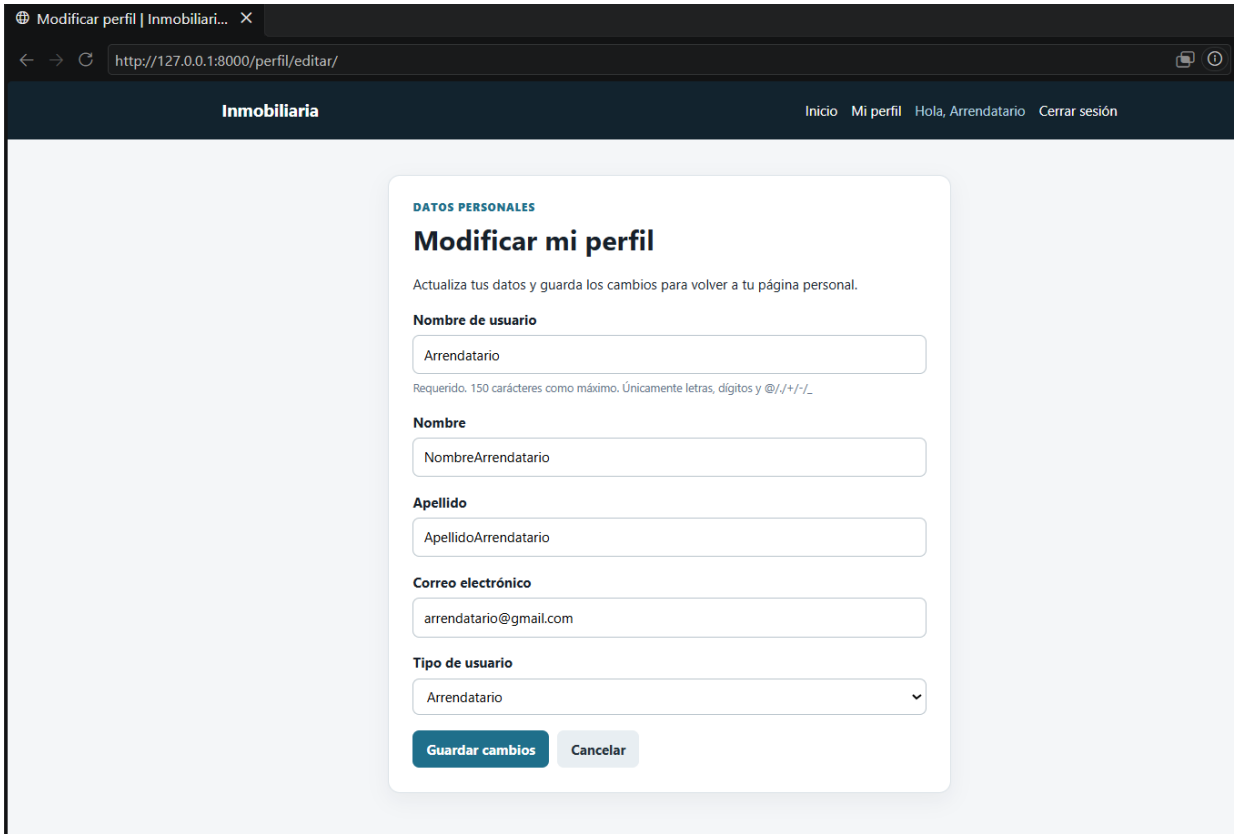
El sitio incorpora una página personal de perfil para usuarios autenticados. En ella se despliegan sus datos personales, tipo de usuario y grupos asociados. También se agregó una pantalla para modificar datos personales.



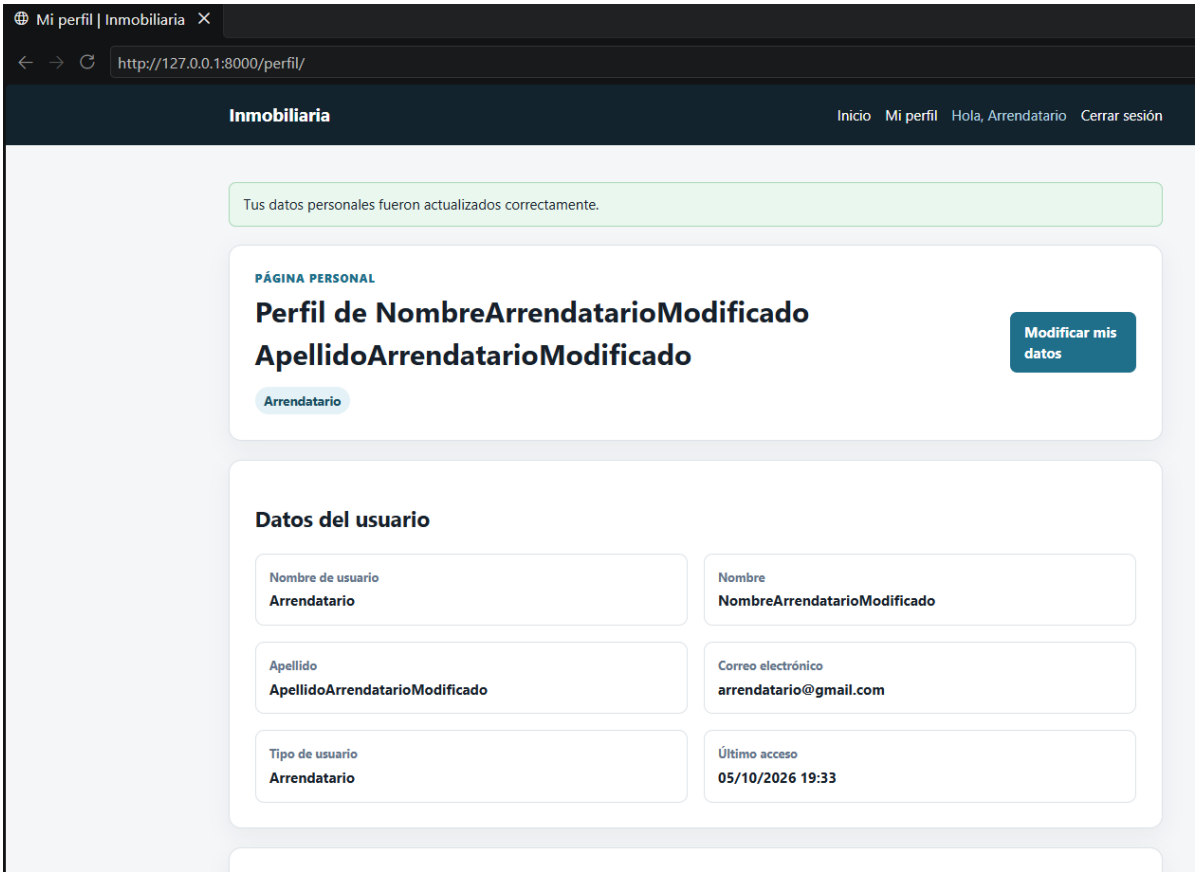
Perfil de Arrendatario con datos del usuario.



Perfil de Arrendador con tipo de usuario asignado.



Formulario para modificar datos personales.



Perfil actualizado luego de guardar cambios.

10. CRUD web de inmuebles con ORM

En la etapa final del desarrollo se incorporó la administración de inmuebles desde páginas web. Esta funcionalidad utiliza formularios basados en modelos y consultas con el ORM de Django.

- Crear inmueble: el arrendador completa un formulario y el sistema guarda el objeto asignando automáticamente el propietario.
- Listar inmuebles propios: cada arrendador visualiza solo las propiedades que le pertenecen.
- Actualizar inmueble: el formulario se carga con datos existentes y guarda los cambios con el ORM.
- Eliminar inmueble: se muestra confirmación antes de borrar el registro.
- Oferta disponible: los arrendatarios revisan el listado de inmuebles publicados.

Ruta	Función
/inmuebles/mis-inmuebles/	Listado de inmuebles del arrendador autenticado.
/inmuebles/agregar/	Formulario para crear un nuevo inmueble.
/inmuebles/<id>/editar/	Formulario para actualizar un inmueble existente.
/inmuebles/<id>/eliminar/	Confirmación y eliminación del inmueble.
/inmuebles/oferta/	Listado de oferta disponible para usuarios autenticados.

11. Demostración funcional sugerida

Para presentar el sitio, se recomienda ejecutar el servidor local y recorrer el flujo funcional completo. La demostración permite evidenciar que la solución no es solo código, sino una aplicación usable.

11. Ejecutar el servidor con python manage.py runserver.
12. Iniciar sesión con un usuario arrendador.
13. Crear un inmueble nuevo desde /inmuebles/agregar/.
14. Editar el inmueble creado y confirmar que los datos cambian.
15. Eliminar un inmueble desde la pantalla de confirmación.
16. Iniciar sesión como arrendatario y revisar la oferta disponible.
17. Mostrar reportes por comuna y región generados en archivos de texto.

12. Problemas encontrados y soluciones aplicadas

Problema	Solución aplicada
Conexión PostgreSQL en Windows	El driver psycopg requería encontrar libpq.dll. Se solucionó agregando temporalmente C:\Program Files\PostgreSQL\18\bin al PATH de la terminal.
Bloqueo de ejecutables por Windows	Algunos comandos como pip.exe fueron bloqueados por Control de aplicaciones. Se utilizó python -m pip para instalar dependencias.
Persistencia de variables de entorno	El PATH temporal se perdía al abrir una nueva terminal. Se documentó el comando necesario antes de ejecutar Django.
Separación de roles	Se diferenciaron Arrendatarios y Arrendadores mediante grupos de Django y lógica de vistas protegidas.
Ejecución de scripts externos	Los scripts de reportes necesitaban cargar la raíz del proyecto en sys.path antes de llamar django.setup().
Seguridad de registros	Las operaciones de edición y eliminación de inmuebles se restringieron al propietario autenticado.

13. Conclusiones

El proyecto cumple el objetivo de construir una aplicación web basada en Django para publicar, administrar y consultar inmuebles disponibles para arriendo. La solución integra PostgreSQL, modelos relacionales, carga de datos, reportes, autenticación, perfiles de usuario, permisos y operaciones CRUD con ORM.

- La estructura del proyecto permite seguir ampliando funcionalidades, por ejemplo filtros visuales por comuna o región desde la oferta.
- El uso de Django Admin aceleró la gestión interna de datos.
- Los formularios basados en modelos facilitaron la creación y actualización de inmuebles.
- La separación por grupos permite extender permisos y reglas de negocio en futuras iteraciones.

14. Anexo: lista de evidencias incluidas

- Admin de Django personalizado con modelos de gestión inmobiliaria.
- Registro, login, logout, grupos y permisos de usuarios.
- Carga de datos con fixtures y verificación en Admin.
- Reportes de inmuebles por comuna y región.
- Perfiles de Arrendatario y Arrendador.
- Formulario de edición de datos personales.
- Resumen del CRUD web de inmuebles con ORM.